

APLIKACJE WEBOWE

Conflicts, Review & PRs

L5 

by ***Mateusz Węglewski***

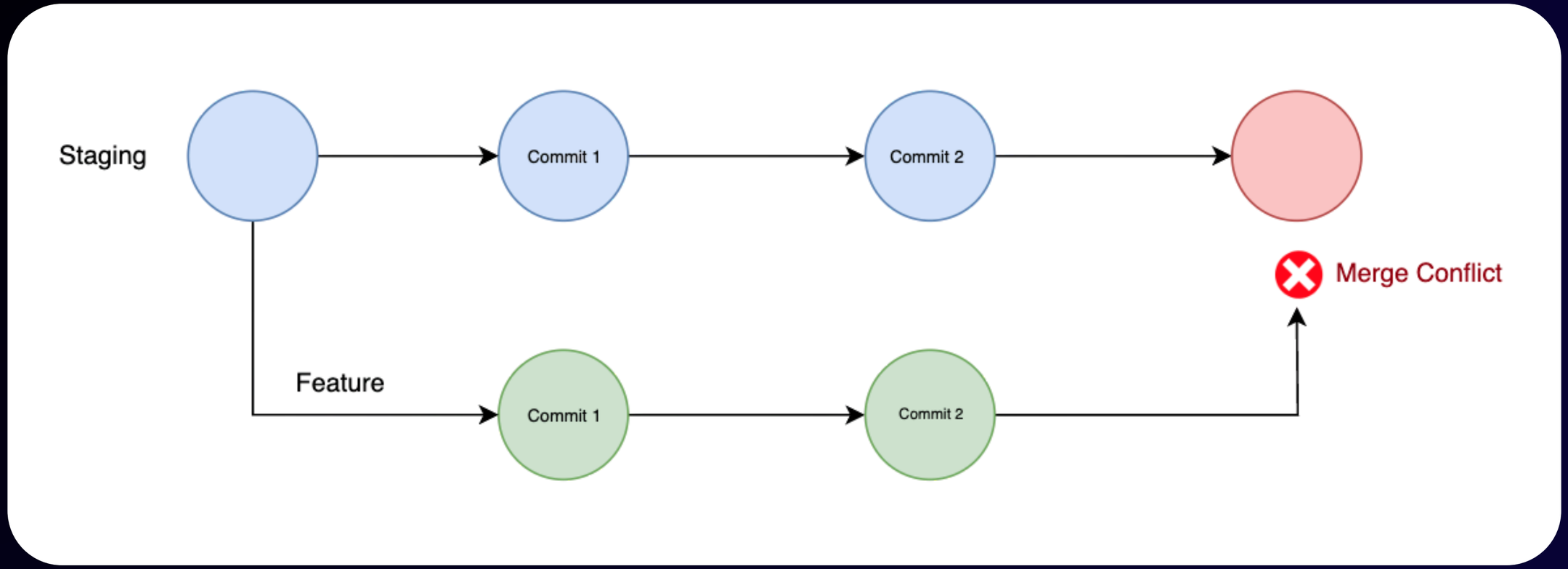


APLIKACJE WEBOWE

Merge conflict

A **merge conflict** happens when two branches change the same part of a file. Git can't decide which change to keep, so you have to choose. You must resolve the conflict before you can finish the merge.




Staging
Commit 1
Commit 2
Merge Conflict
Feature
Commit 1
Commit 2

```
<<<<<<< HEAD
...treść z Twojej bieżącej gałęzi...
=======
...treść z gałęzi, którą scalasz...
>>>>>>> nazwa-gałęzi
```

HEAD to zawsze Twoja strona; treść pod === to strona przychodząca.

by **Mateusz Węglewski**



APLIKACJE WEBOWE

Konflikt TO NIE BŁĄD

Konflikty są nieuniknione

Gdy dwie osoby zmieniają tę samą linię kodu, Git potrzebuje człowieka, by zdecydować, co zostaje.

Historia ma znaczenie

To, czy używasz merge czy rebase, wpływa na to, jak czytelny będzie log projektu za pół roku.

Review chroni wszystkich

Nawet najlepszy programista czasem się myli - drugie spojrzenie łapie to, czego autor nie widzi.



by ***Mateusz Węglewski***



APLIKACJE WEBOWE

REMINDER

Merge

git merge is a Git command used to combine changes from two different development lines (called branches) into a single branch.
Think of it like two people working on different chapters of the same book; merging is the act of putting those chapters back together into the main manuscript.



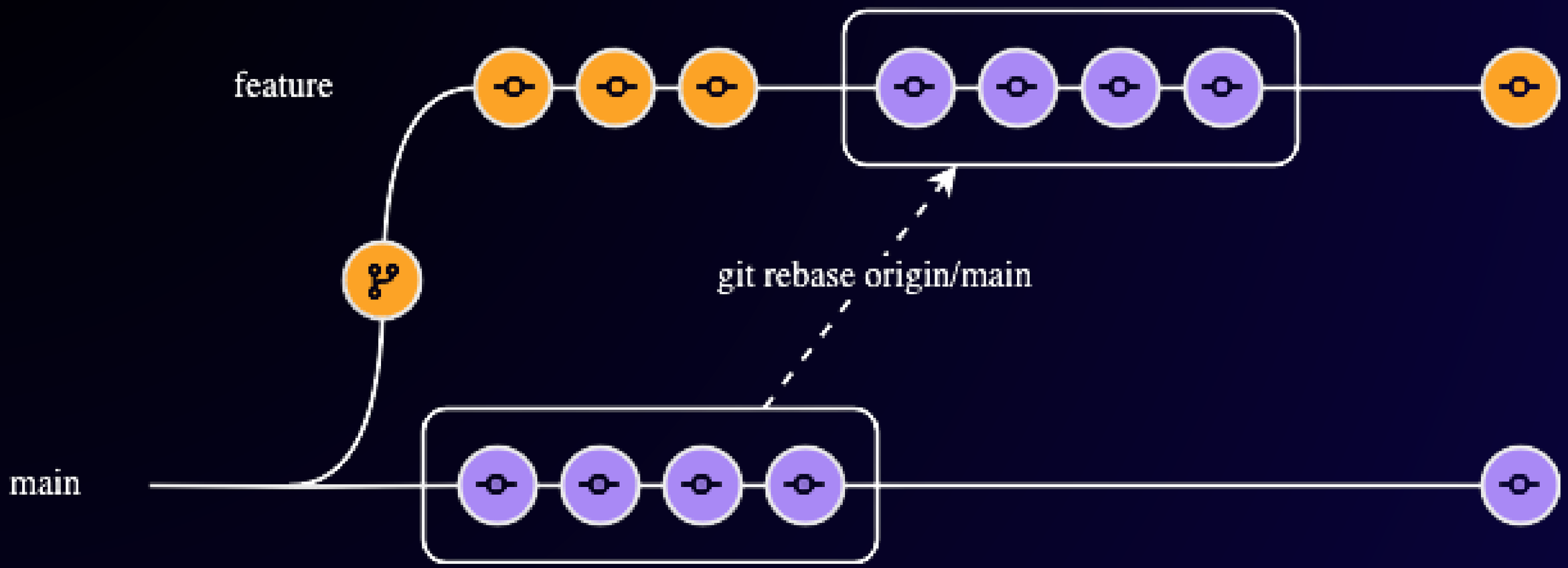


APLIKACJE WEBOWE

Rebase

git rebase is a command that moves or combines a sequence of commits to a new base commit. It rewrites project history by applying your local changes on top of another branch.





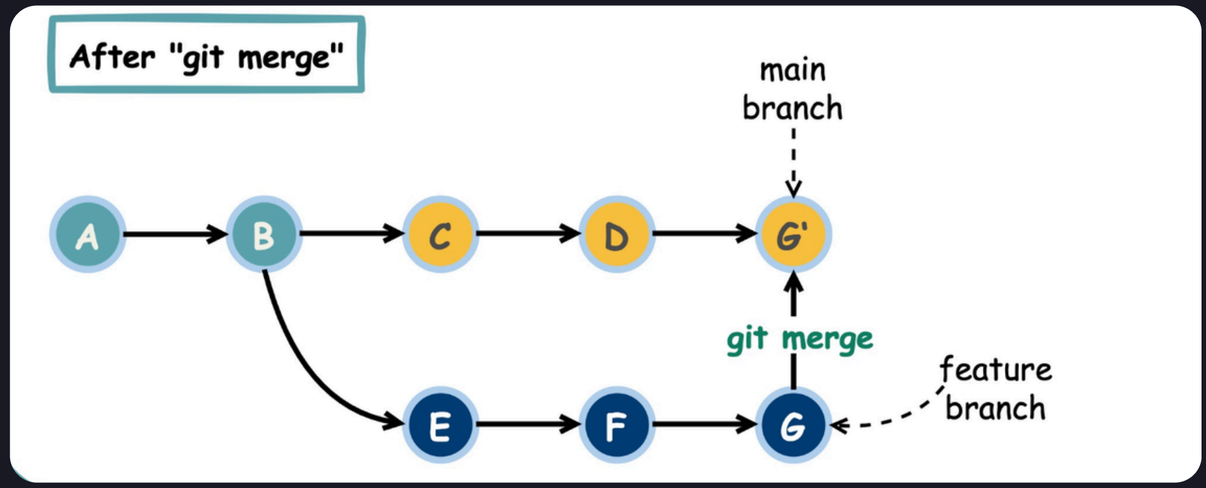
by **Mateusz Węglewski**

git merge zachowuje historię rozgałęzienia

```
git switch main
git merge feature

Merge made by the 'recursive' strategy.
Merge branch 'feature'

# historia: main i feature widoczne osobno,
# połączone jednym commitem scalającym
```



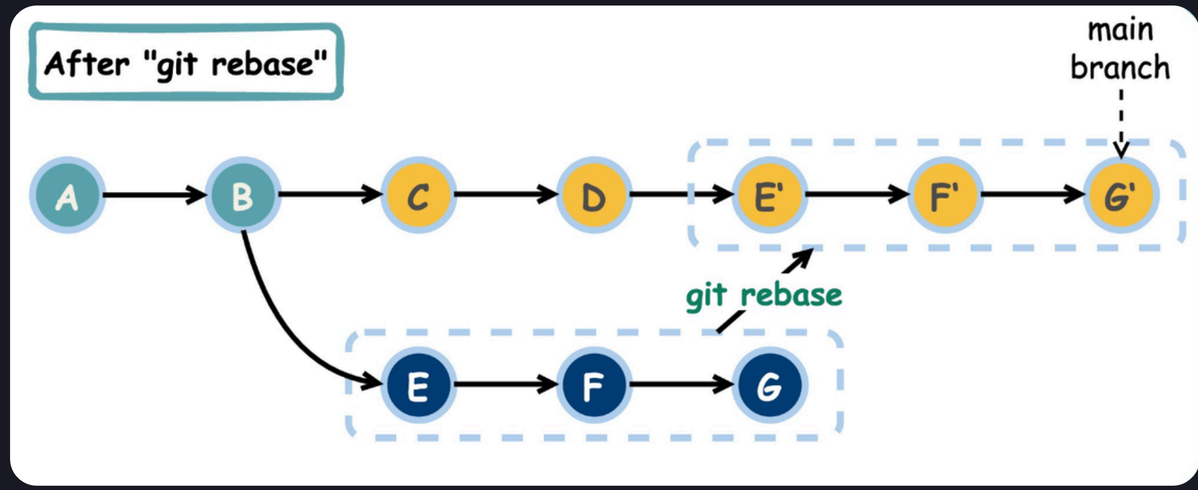
by **Mateusz Węglewski**

git rebase przepisuje historię na liniową

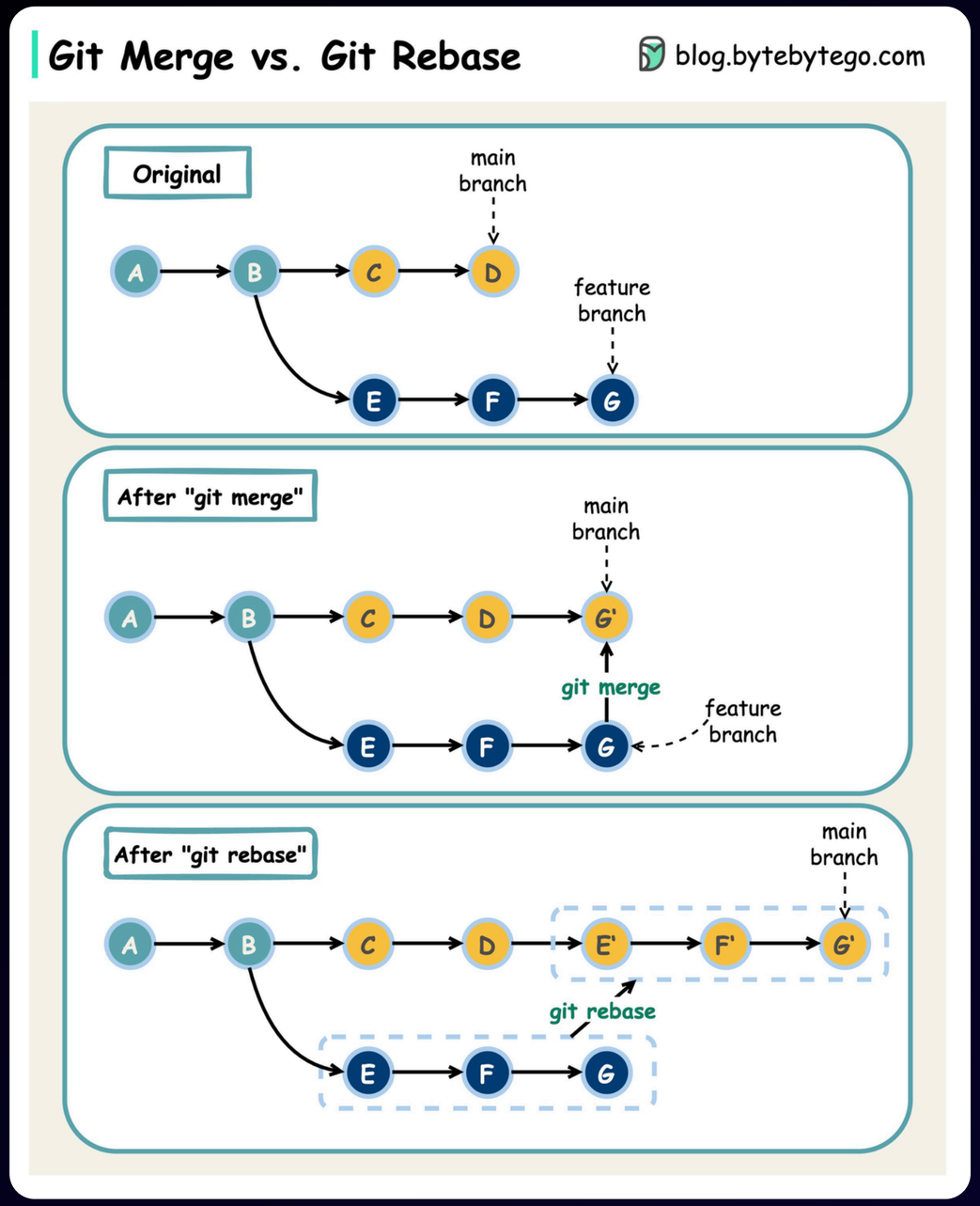
```
git switch feature
git rebase main

# commity z feature zostają 'przełożone'
# na czubek main, jeden po drugim

git switch main
git merge feature   # teraz to fast-forward
```



by **Mateusz Węglewski**



Git squash

Fun Fact

Git squash is the process of combining multiple commits into a single one. It keeps your repository history clean and readable before merging changes into the main branch.

```
Historia
rozgałęziona

Widać dokładnie,
kiedy i co
zostało scalone
```

```
Historia
liniowa

Wygląda, jakby
praca powstała
jedna po drugiej
```


by **Mateusz Węglewski**



In practice

APLIKACJE WEBOWE

rebase — lokalnie, przed PR	**porządkujesz własną, jeszcze niewypchniętą pracę — czysta, liniowa historia**
merge — gałęzie współdzielone	**gdy gałąź już wypchnięta i inni na niej pracują — rebase zmieniłby ich historię**

NIGDY NIE REBASE'UJ COMMITÓW, KTÓRE JUŻ WYPCHNĄŁEŚ I KTÓRE KTOŚ INNY MÓGŁ POBRAĆ



Merge conflict

DIAGNOSTYKA

1. **`git status`**
   pokazuje, który plik ma nierozwiązany konflikt
2. **`otwórz plik`**
   zdecyduj, którą wersję (lub połączenie) zachować; usuń wszystkie trzy znaczniki
3. **`git add <plik>`**
   oznacza konflikt jako rozwiązany
4. **`git commit`**
   kończy scalanie; Git proponuje gotową wiadomość

by *Mateusz Węglewski*

Merge conflict

GDY COŚ SIĘ ZEPSUJE

```
git merge --abort
```

Cofa scalanie do stanu sprzed jego rozpoczęcia — możesz zacząć od nowa. Bezpieczna operacja, warto ją znać zanim ktoś zacznie panikować.

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

"There is no perfect code –
there is only better code."



by ***Mateusz Węglewski***

Checklista reviewera

1. **`funkcjonalność`**
   czy kod robi to, co powinien i jest dobry dla użytkowników
2. **`złożoność`**
   czy nie jest bardziej skomplikowany niż musi być na tym etapie
3. **`testy`**
   czy zmiana ma odpowiednie, sensowne testy
4. **`nazewnictwo i komentarze`**
   czy nazwy są jasne, a komentarze tłumaczą 'dlaczego', nie 'co'

by ***Mateusz Węglewski***

✕ Unikaj

"To jest źle."

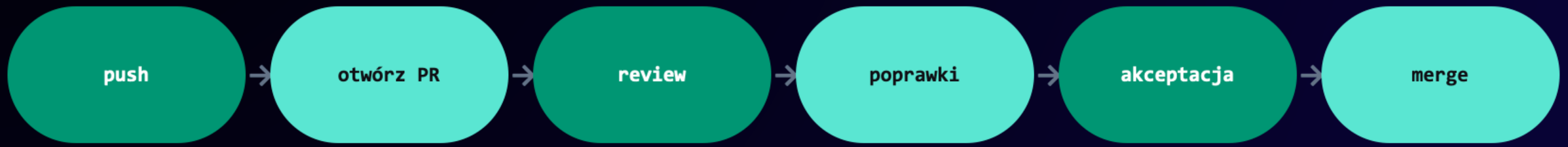
"Zła nazwa."

✓ Zamiast tego

"Could you clarify...?"

"Consider renaming..."

branch – merge cycle



"poprawki" i "review" mogą się powtórzyć kilka razy, zanim PR zostanie zaakceptowany.

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Polecenie	Co robi
`git rebase <gałąź>`	przekłada commity bieżącej gałęzi na czubek wskazanej gałęzi — tworzy liniową historię
`git rebase --abort`	przerywa rebase i cofa do stanu sprzed jego rozpoczęcia
`git rebase --continue`	kontynuuje rebase po ręcznym rozwiązaniu konfliktu na danym kroku
`git merge <gałąź>`	scala wskazaną gałąź do bieżącej — może zgłosić konflikt
`git status`	pokazuje, który plik ma nierozwiązany konflikt
`git add <plik>`	oznacza konflikt w danym pliku jako rozwiązany
`git commit`	(bez -m) kończy scalanie — Git otwiera edytor z gotową wiadomością
`git merge --abort`	koło ratunkowe — cofa scalanie do stanu sprzed konfliktu
`git init --bare`	tworzy repozytorium pełniące rolę zdalnego "serwera" zespołu
`git push origin <gałąź>`	wysyła commity danej gałęzi do zdalnego repo
`git fetch origin`	pobiera informacje o zmianach na zdalnym repo (bez scalania)
`git diff <gałąź-A> <gałąź-B>`	pokazuje różnicę między dwiema gałęziami — podstawa code review





APLIKACJE WEBOWE

:wq

L5



by ***Mateusz Węglewski***